

Machine Learning

Outline

- Introduction to Machine Learning , Concepts of Learning, Supervised, Unsupervised and Reinforcement Learning,
- Statistical-based Learning: Naive Bayes Model,
- Learning by Genetic Algorithms: Operators in Genetic Algorithm: Selection, Mutation, Crossover, Fitness Function, Genetic Algorithm,
- Learning with Neural Networks: Introduction, Biological Neural Networks Vs. Artificial Neural Networks (ANN), Mathematical Model of ANN, Activation Functions: Linear, Step Sigmoid,
- Types of ANN: Feed-forward, Recurrent, Single Layered, Multi-Layered, Application of Artificial Neural Networks, Learning by Training ANN, Hebbian Learning, Perceptron Learning, Back-propagation Learning,
- Overview of Deep Learning Architectures : Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Generative Adversarial Networks (GANs).

What is learning ?

- Learning is the process of acquiring new or modifying existing knowledge, behaviors, skills, values, or preferences.
- Evidence that learning has occurred may be seen in changes in behavior from simple to complex.

What is Machine learning?

- The term machine learning was first introduced by Arthur Samuel in 1959. We can define it in a summarized way as: **Machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things without being explicitly programmed.**

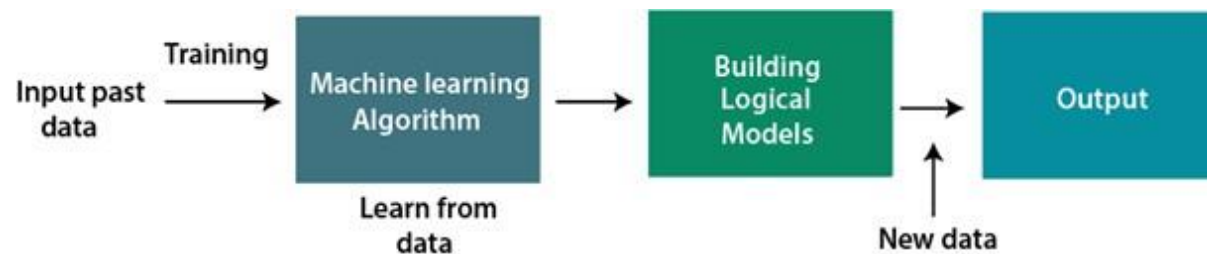
We give below two more definitions.

- **When the agent is a computer, we call it machine learning: a computer observes some data, builds a model based on the data, and uses the model as both a hypothesis about the world and a piece of software that can solve problems.**

- The machine Learning denotes changes in the system that are adaptive in the sense that they enable the system to do the same task more effectively the next time.
- Like human learning from past experiences, computer system learns from data, which represent some “past experiences” of an application domain.
- Therefore, Machine learning gives computers the ability to learn without being explicitly programmed.

How does Machine Learning work

- A machine learning system builds prediction models, learns from previous data, and predicts the output of new data whenever it receives it.
- The amount of data helps to build a better model that accurately predicts the output, which in turn affects the accuracy of the predicted output.
- The Machine Learning algorithm's operation is depicted in the following block diagram:



Features of Machine Learning

- Machine learning uses data to detect various patterns in a given dataset.
- It can learn from past data and improve automatically.
- It is a data-driven technology.
- Machine learning is much similar to data mining as it also deals with the huge amount of the data.

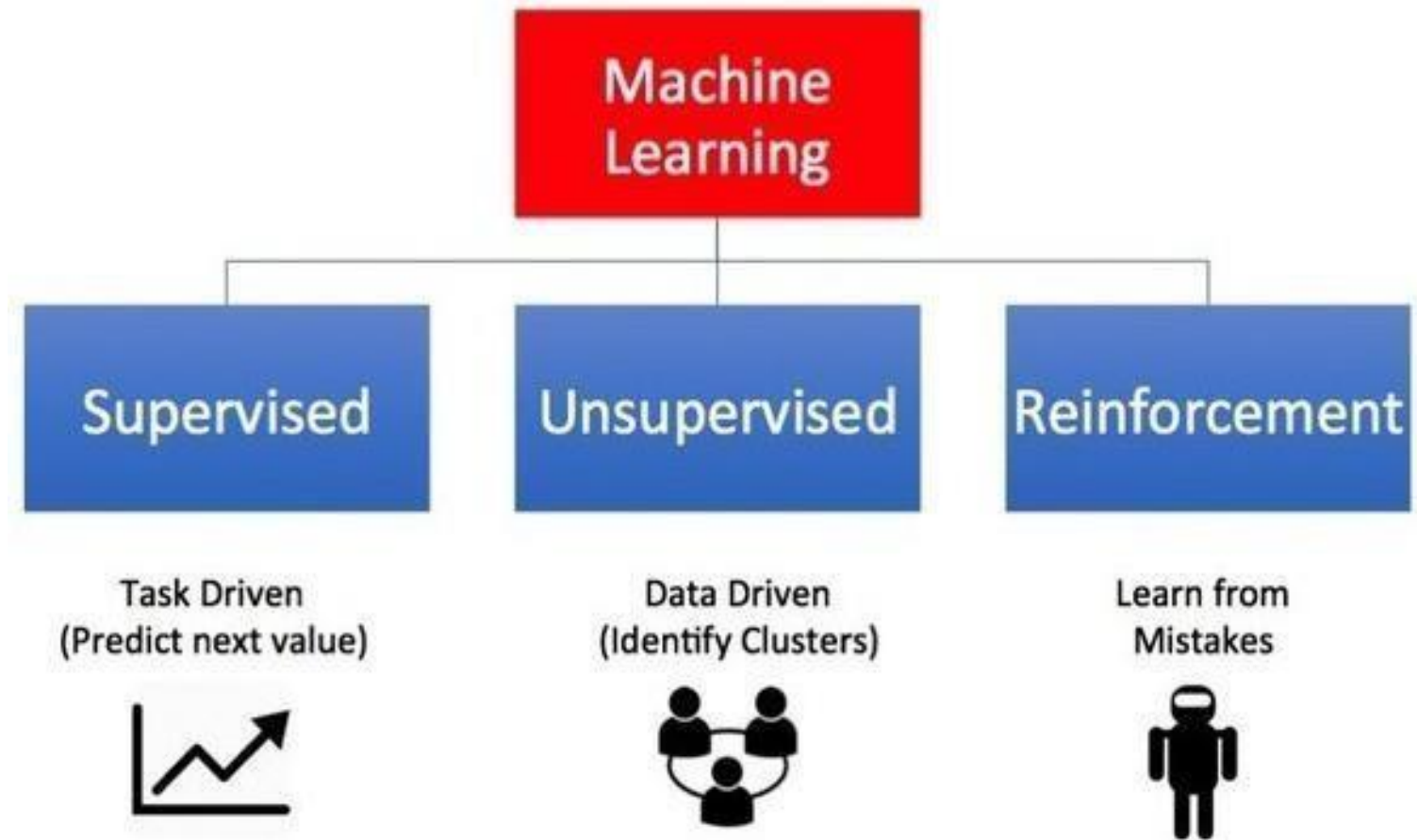
Importance of Machine Learning

- Rapid increment in the production of data.
- Solving complex problems, which are difficult for a human.
- Decision making in various sector including finance.
- Finding hidden patterns and extracting useful information from data.

Types of Learning

In general, machine learning algorithms can be classified into three types.

- Supervised learning
- Unsupervised learning
- Reinforcement learning



Supervised learning

- **Supervised learning** involves training an agent by providing **input-output pairs**.
- The goal is to learn a function that maps inputs to the correct outputs.

Example - Image Classification:

- Inputs: Camera images.
- Outputs (labels): "Bus," "Pedestrian," etc.
- The model learns to **predict labels** for new images.

Example - Braking Actions in a Car:

- Inputs: Speed, direction, road conditions.
- Outputs: Stopping distance.
- The environment acts as a **teacher**, providing feedback.

Applications of Supervised Learning:

- **Risk Assessment** (e.g., credit scoring).
- **Fraud Detection** (e.g., identifying fraudulent transactions).
- **Spam Filtering** (e.g., classifying emails as spam or not).

- **Speech Recognition** : Converting spoken words into text (e.g., voice assistants like Siri or Google Assistant).
- **Weather Prediction** :Predicting tomorrow's temperature based on historical weather data.
- **Stock Market Prediction**: Predicting stock prices based on labeled historical stock performance.

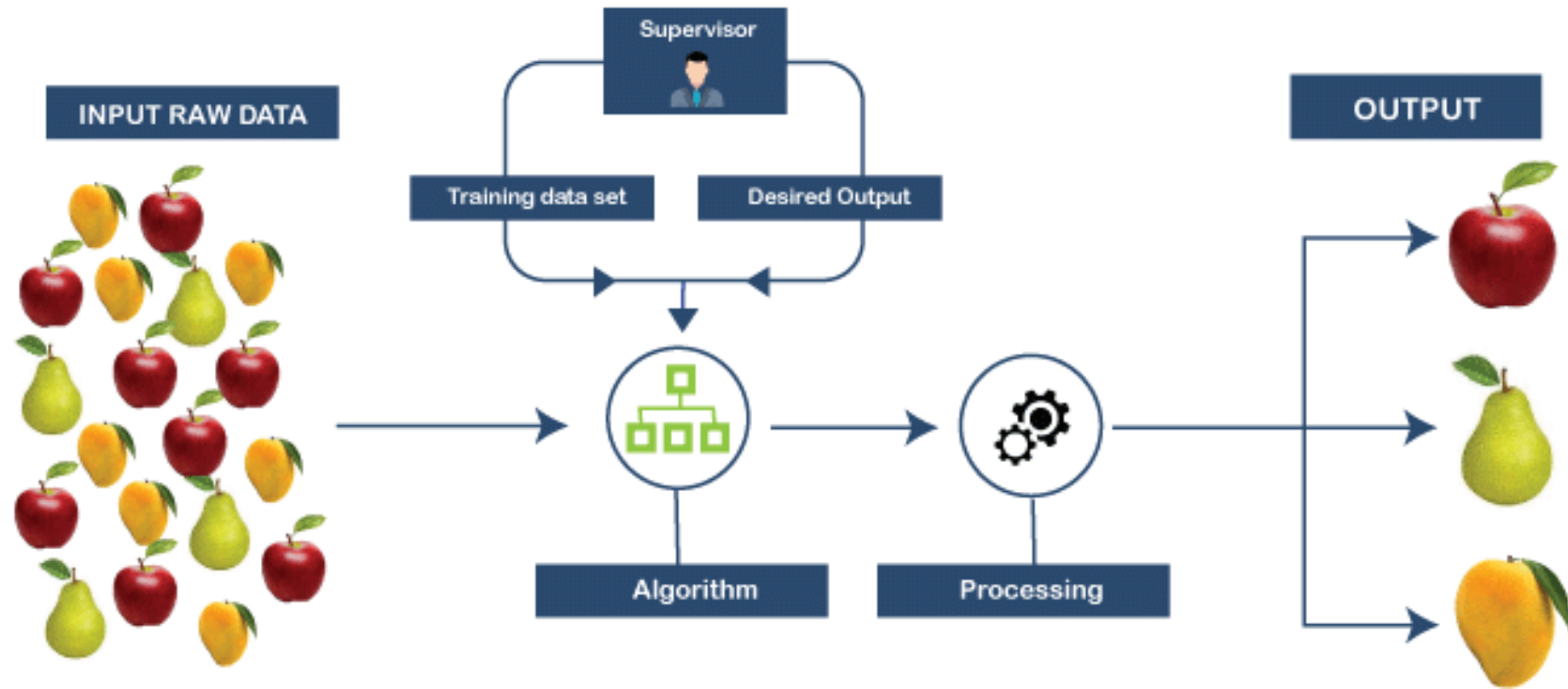
The main goal of the supervised learning technique is to map the input variable(x) with the output variable(y).

- Examples of supervised learning:

Input (x)	Output (y)	Application
Home features	Price	Real Estate
Ad, user info	Click on ad? (0/1)	Online Advertising
Image	Object classification (1,...,1000)	Photo Tagging
Audio	Text transcript	Speech Recognition
English text	Chinese text	Machine Translation
Image, Radar info	Position of other cars	Autonomous Driving

Working Mechanism

SUPERVISED LEARNING



- **Supervised learning** is named this way because the learning process resembles a **teacher-student** relationship:

Labeled Data: The training dataset consists of **input-output pairs** where the **correct output (label)** is already known.

Prediction & Correction: The algorithm makes predictions and is corrected based on the **difference** between its prediction and the actual label.

Iterative Learning: The model **adjusts** its parameters (e.g., weights in a neural network) over multiple iterations to improve accuracy.

Stopping Condition: Learning **stops** when the model achieves an acceptable level of performance, such as **low error** or **high accuracy** on the training data.

Example:

To predict whether a new patient is **healthy or sick** based on their **age** and **gender**, we can use a **supervised learning classification model**. Here's how: **Input Features (X):** Gender, Age, **Output (Y):** Health Status (Healthy/Sick)

Gender	Age	Health Status (Label)
Male	25	Healthy
Female	30	Healthy
Male	50	Sick
Female	65	Sick
Male	40	Healthy
Female	55	Sick

Based on this data, when a new patient enters the clinic, how can one predict whether he/she is healthy or sick?

Supervised learning can be grouped further in two categories of algorithms:

Classification : Classification algorithms are used to solve the classification problems in which the output variable is categorical, such as "Yes" or No, Male or Female, Red or Blue, etc.

- Some real-world examples of classification algorithms are Spam Detection, Email filtering, etc.
- **Some popular classification algorithms are given below:**
 - Logistic regression
 - Naive Bayes algorithm
 - K-NN algorithm
 - Decision tree algorithm
 - Support vector machine algorithm
 - Random forest algorithm

- **Regression :** Regression algorithms are used to solve regression problems in which there is a linear relationship between input and output variables.
- These are used to predict continuous output variables, such as market trends, weather prediction, etc.

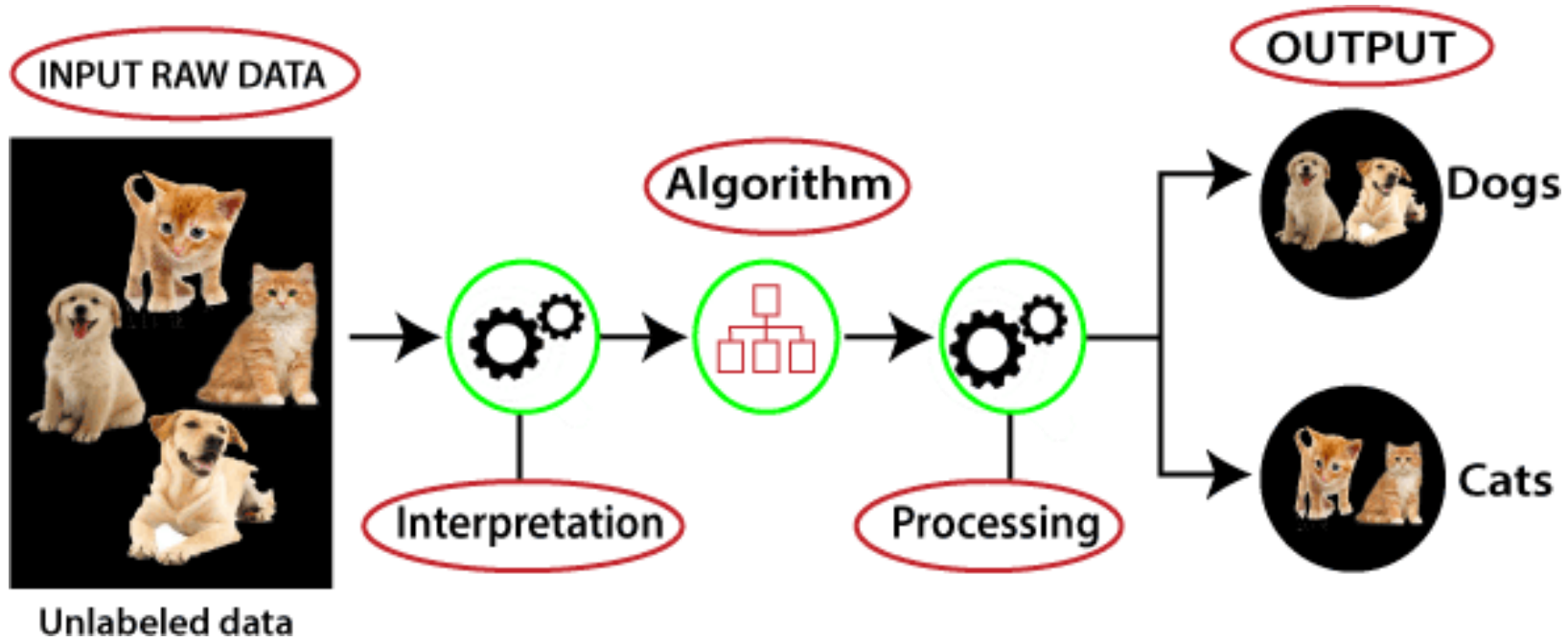
Some popular Regression algorithms are given below:

- Linear Regression
- Regression Trees
- Non-Linear Regression
- Bayesian Linear Regression
- Polynomial Regression

Unsupervised learning

- Used to draw inferences from datasets consisting of input data without labeled responses.
- In unsupervised learning algorithms, a classification or categorization is not included in the observations. There are no output values and so there is no estimation of functions. Since the examples given to the learner are unlabeled, the accuracy of the structure that is output by the algorithm cannot be evaluated
- The most common unsupervised learning task is **clustering**: detecting potentially useful clusters of input examples.
- For example, when shown millions of images taken from the Internet, a computer vision system can identify a large cluster of similar images which an English speaker would call “cats.”
- **The main aim of the unsupervised learning algorithm is to group or categories the unsorted dataset according to the similarities, patterns, and differences.**

Working Mechanism



Types of Unsupervised Learning Algorithm:

- **Clustering:** Clustering is a method of grouping the objects into clusters such that objects with most similarities remain into a group and have less or no similarities with the objects of another group.
- **Association:** An association rule is an unsupervised learning method which is used for finding the relationships between variables in the large database. Such as people who buy X item (suppose a bread) also tend to purchase Y (Butter/Jam) item. A typical example of Association rule is Market Basket Analysis.
- Below is the list of some popular unsupervised learning algorithms:
 - K-means clustering, KNN (k-nearest neighbors)
 - Hierarchical clustering, Anomaly detection
 - Neural Networks, Principle Component Analysis
 - Independent Component Analysis, Apriori algorithm
 - Singular value decomposition

Applications of Unsupervised Learning Algorithms

- **Market Basket Analysis:** Identifying products frequently bought together.
- **Customer Segmentation:** Grouping customers based on purchasing behavior.
- **Anomaly Detection:** Detecting fraudulent transactions or network intrusions.
- **Data Preprocessing:** Reducing dimensions and noise in data for further analysis.
- **Image and Speech Recognition:** Extracting features from raw data for classification tasks.

Reinforcement learning

- An **agent learns from a series of reinforcements** in the form of **rewards and punishments**.
- It mimics how humans or animals learn from experience by interacting with an environment and adjusting actions based on feedback.
- Example: In a **chess game**, the agent is informed whether it has **won (reward)** or **lost (punishment)**. It then evaluates which actions led to the outcome and modifies its future strategy to maximize rewards.
- Another example, Consider teaching a dog a new trick. Since we cannot explicitly tell it what to do, we use rewards (treats) for correct behavior and punishments (lack of treats, verbal correction) for incorrect behavior. Over time, the dog learns the correct actions to maximize rewards
- We can use a similar method to train computers to do many tasks, such as playing video games, chess, scheduling jobs, and controlling robot limbs.
- The reinforcement learning approach in machine learning is to determine the **best sequence of actions** that maximizes **long-term rewards** in a given environment.

Reinforcement Learning in ML



Applications of Reinforcement Learning

- Playing video games (e.g., AlphaGo, OpenAI Five)
- Chess-playing AI (e.g., Stockfish, AlphaZero)
- Job scheduling (e.g., optimizing task execution in cloud computing)
- Robotics (e.g., robotic arms learning to pick and place objects)

Naïve Bayes Classifier

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
- It is mainly used in *text classification* that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object.**
- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles.**

Why is it called Naïve Bayes?

- **Naïve:** It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple.
- Hence each feature individually contributes to identify that it is an apple without depending on each other.
- **Bayes:** It is called Bayes because it depends on the principle of **Bayes' Theorem**.

Bayes' Theorem:

- Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
- The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where,

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

P(A) is Prior Probability: Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability: Probability of Evidence.

Working of Naïve Bayes' Classifier:

Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**". So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions.

So, to solve this problem, we need to follow the below steps:

- Convert the given dataset into frequency tables.
- Generate Likelihood table by finding the probabilities of given features.
- Now, use Bayes theorem to calculate the posterior probability.

Problem: If the weather is sunny, then the Player should play or not?

Solution: To solve this, first consider the below dataset:

Days	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	5

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

$$P(\text{Yes}|\text{Sunny}) = P(\text{Sunny}|\text{Yes}) * P(\text{Yes}) / P(\text{Sunny})$$

$$P(\text{Sunny}|\text{Yes}) = 3/10 = 0.3$$

$$P(\text{Sunny}) = 0.35$$

$$P(\text{Yes}) = 0.71$$

$$\text{So, } P(\text{Yes}|\text{Sunny}) = 0.3 * 0.71 / 0.35 = \mathbf{0.60}$$

$$P(\text{No}|\text{Sunny}) = P(\text{Sunny}|\text{No}) * P(\text{No}) / P(\text{Sunny})$$

$$P(\text{Sunny}|\text{NO}) = 2/4 = 0.5$$

$$P(\text{No}) = 0.29$$

$$P(\text{Sunny}) = 0.35$$

$$\text{So, } P(\text{No}|\text{Sunny}) = 0.5 * 0.29 / 0.35 = \mathbf{0.41}$$

So as we can see from the above calculation that **$P(\text{Yes}|\text{Sunny}) > P(\text{No}|\text{Sunny})$**

Introduction to Genetic Algorithms(GA)

- Nature has always been a great source of inspiration to all mankind.
- A genetic algorithm is a search heuristic that is inspired by Charles Darwin's theory of natural evolution.

“Select The Best, Discard The Rest”

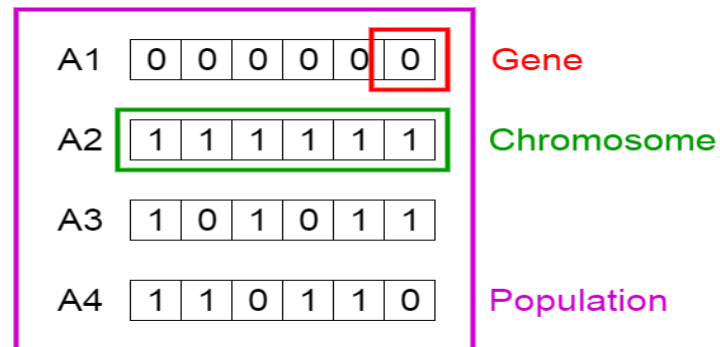
- i.e., the fittest individuals are selected for reproduction in order to produce offspring of the next generation.
- GAs were developed by John Holland and his students and colleagues at the University of Michigan.

Notion of Natural Selection

- The process of natural selection starts with the selection of fittest individuals from a population. They produce offspring which inherit the characteristics of the parents and will be added to the next generation.
- If parents have better fitness, their offspring will be better than parents and have a better chance at surviving.
- This process keeps on iterating and at the end, a generation with the fittest individuals will be found.
- This notion can be applied for a search problem. We consider a set of solutions for a problem and select the set of best ones out of them.

Basic Terminology

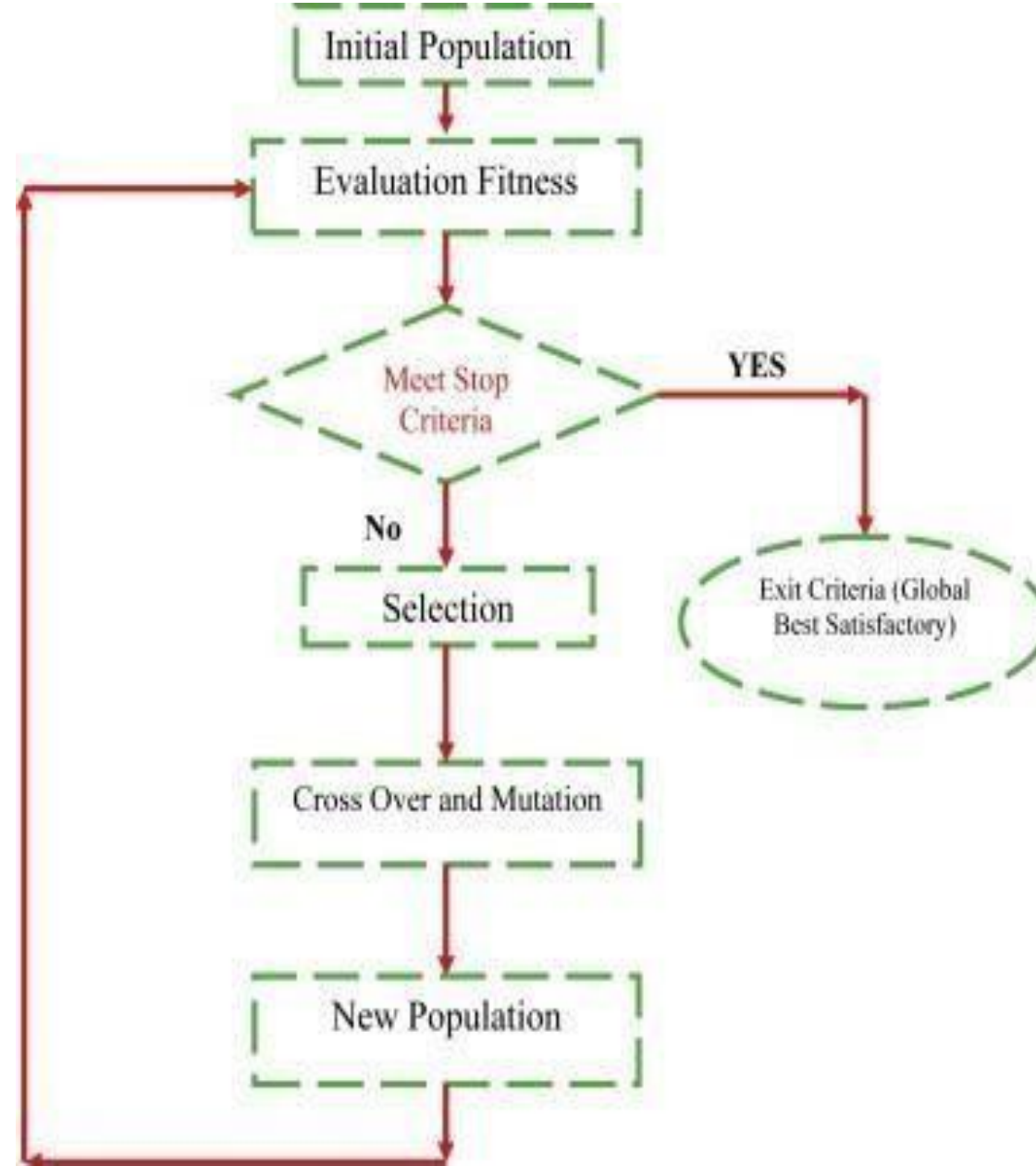
- **Population:** set of individuals each representing a possible solution to a given problem.
- **Gene:** a solution to problem represented as a set of parameters ,these parameters known as genes.
- **Chromosome:** genes joined together to form a string of values called chromosome.
- **Fitness Function:** measures the quality of solution. It is problem dependent.



Nature to Computer Mapping: Biological evolution concepts in nature and their corresponding concepts in computing, particularly in Genetic Algorithms (GAs)

Nature	Computer (Genetic Algorithm Concepts)
Population	Set of solutions
Individual	Solution to a problem
Fitness	Quality of a solution
Chromosome	Encoding for a solution
Gene	Part of the encoding of a solution
Reproduction	Crossover

Five phases are considered in a genetic algorithm



Initialization of Population:

- There are two primary methods to initialize a population in a GA. They are –
- **Random Initialization**
- **Heuristic initialization**
- There are several things to be kept in mind when dealing with GA population –
 - The diversity of the population should be maintained otherwise it might lead to premature convergence.
 - The population size should not be kept very large as it can cause a GA to slow down, while a smaller population might not be enough for a good mating pool. Therefore, an optimal population size needs to be decided by trial and error.

Fitness Function:

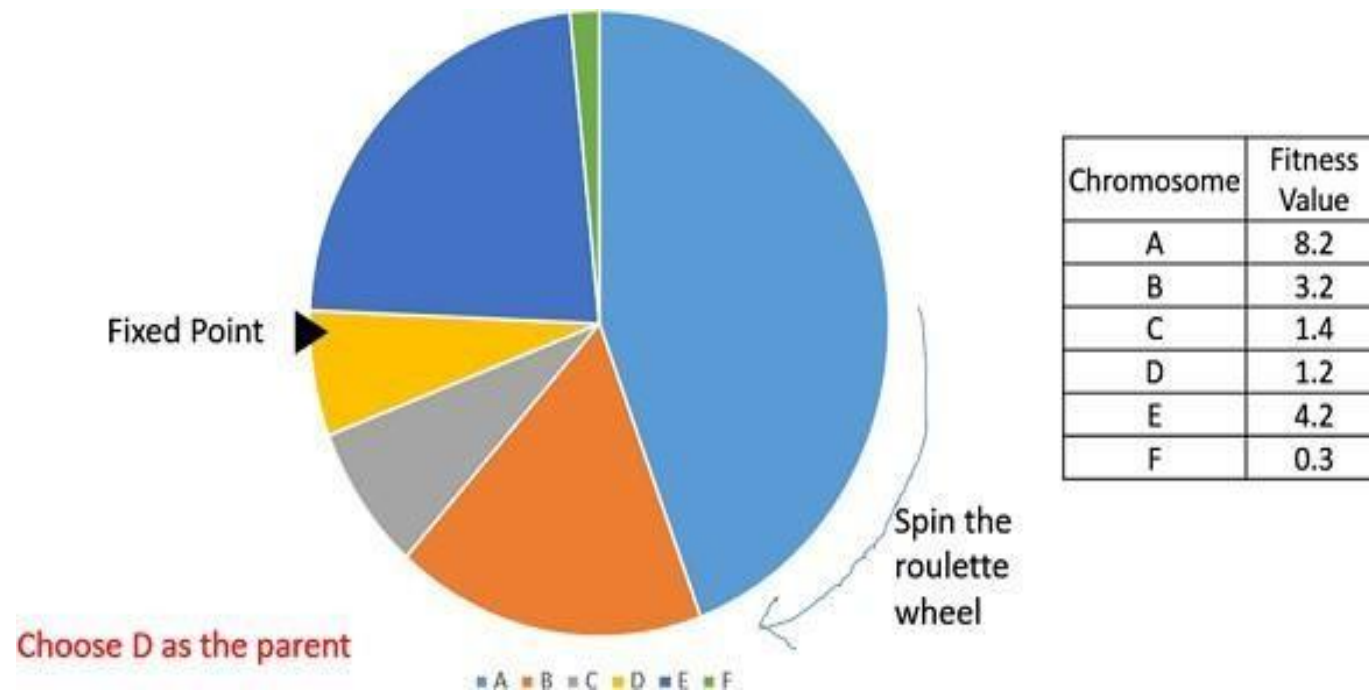
- We have to select the best ones to reproduce offspring out of the available chromosomes, so each chromosome is given a fitness value.
- The fitness score helps to select the individuals who will be used for reproduction.
- A fitness function should possess the following characteristics –
 - The fitness function should be sufficiently fast to compute.
 - It must quantitatively measure how fit a given solution is or how fit individuals can be produced from the given solution.

Selection:

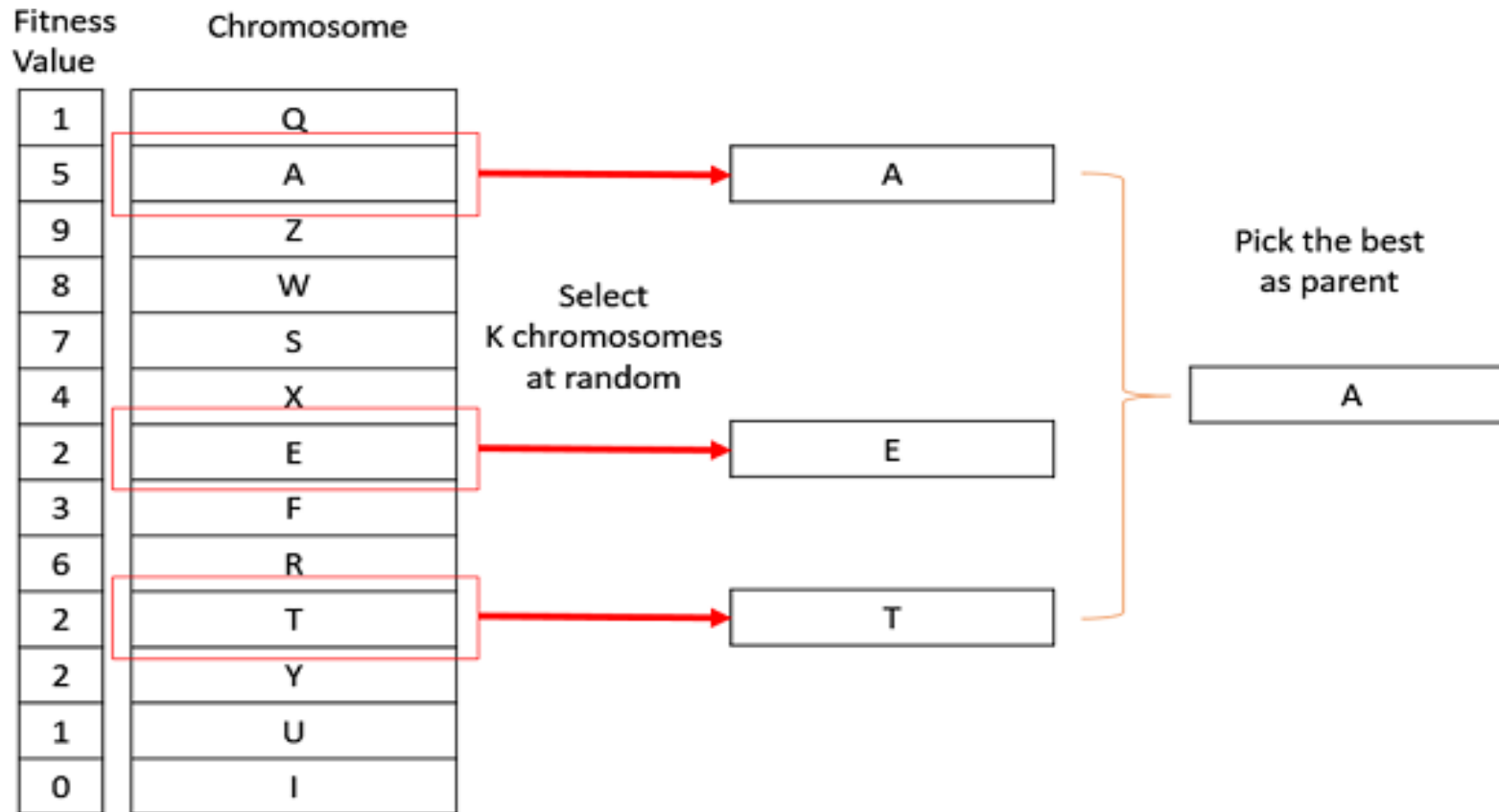
- The idea of selection phase is to select the fittest individuals and let them pass their genes to the next generation.
- Two pairs of individuals (parents) are selected based on their fitness scores. Individuals with high fitness have more chance to be selected for reproduction.
- Selection of the parents can be done by using any one of the following strategy:
 - Fitness Proportionate Selection (Roulette Wheel Selection)
 - Tournament Selection
 - Rank Selection

- **Fitness Proportionate Selection(Roulette Wheel Selection):**

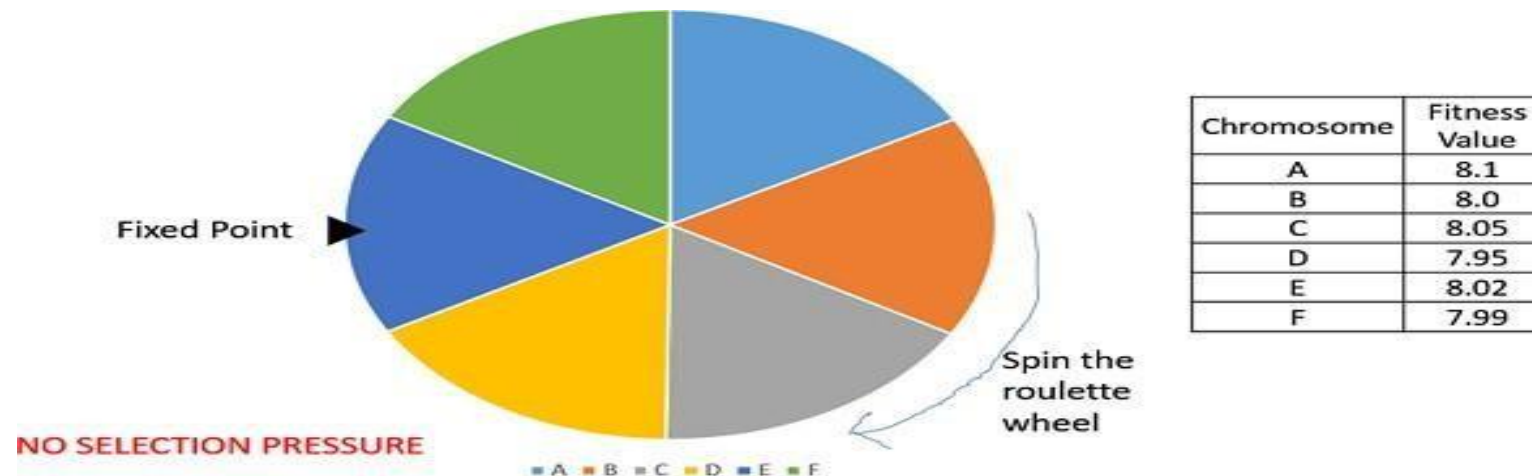
- Chance of selection is directly propositional to the fitness value. This is chance not fixed so, any one can become parent.



- **Tournament Selection:** Take randomly k individuals and select only one among k to make the parent of the next generation.



- **Rank Selection:** mostly used when the individuals in the population have very close fitness values. This leads to each individual having an almost equal share of the pie chart. and hence each individual no matter how fit relative to each other has an approximately same probability of getting selected as a parent. This in turn leads to a loss in the selection pressure towards fitter individuals, making the GA to make poor parent selections in such situations.

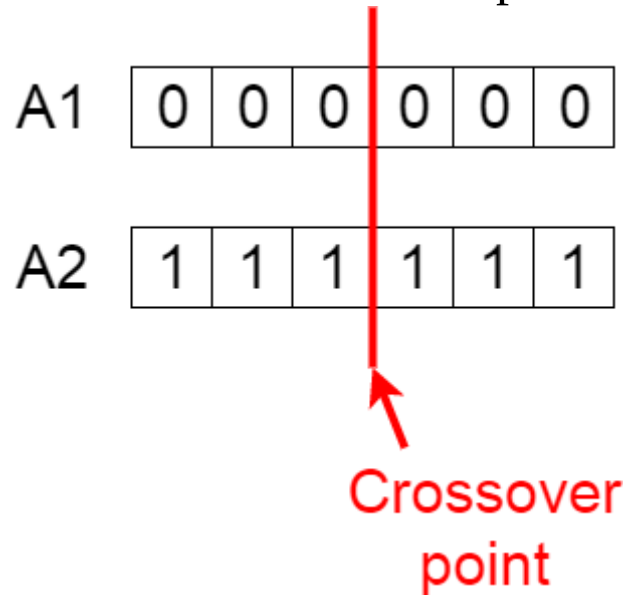


- Every individual in the population is ranked according to their fitness. The higher ranked individuals are preferred more than the lower ranked ones.

Chromosome	Fitness Value	Rank
A	8.1	1
B	8.0	4
C	8.05	2
D	7.95	6
E	8.02	3
F	7.99	5

Crossover:

- Crossover is the most significant phase in a genetic algorithm. For each pair of parents to be mated, a crossover point is chosen at random from within the genes.
- For example, consider the crossover point to be 3 as shown below.



- Offspring are created by exchanging the genes of parents among themselves until the crossover point is reached.



- The new offspring are added to the population.

Mutation:

- Now if you think in the biological sense, are the children produced have the same traits as their parents? The answer is NO. During their growth, there is some change in the genes of children which makes them different from its parents.
- This process is known as mutation, which may be defined as a random tweak in the chromosome.
- This implies that some of the bits in the bit string can be flipped.



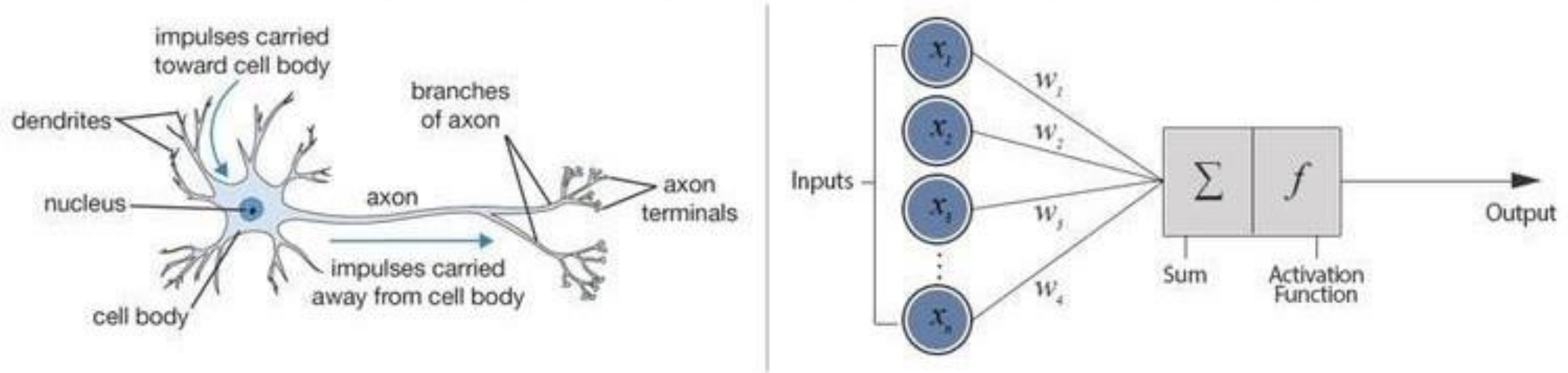
- Mutation occurs to maintain diversity within the population and prevent from the premature convergence.

Learning with Neural Networks

- Neural networks are computational models inspired by the human brain's structure and function.
- They consist of interconnected nodes (neurons) organized in layers, which process information and learn to perform tasks by adjusting the connections (weights) between neurons based on input data.

Biological Neural Network(BNN) vs. Artificial neural network(ANN)

Biological Neuron versus Artificial Neural Network



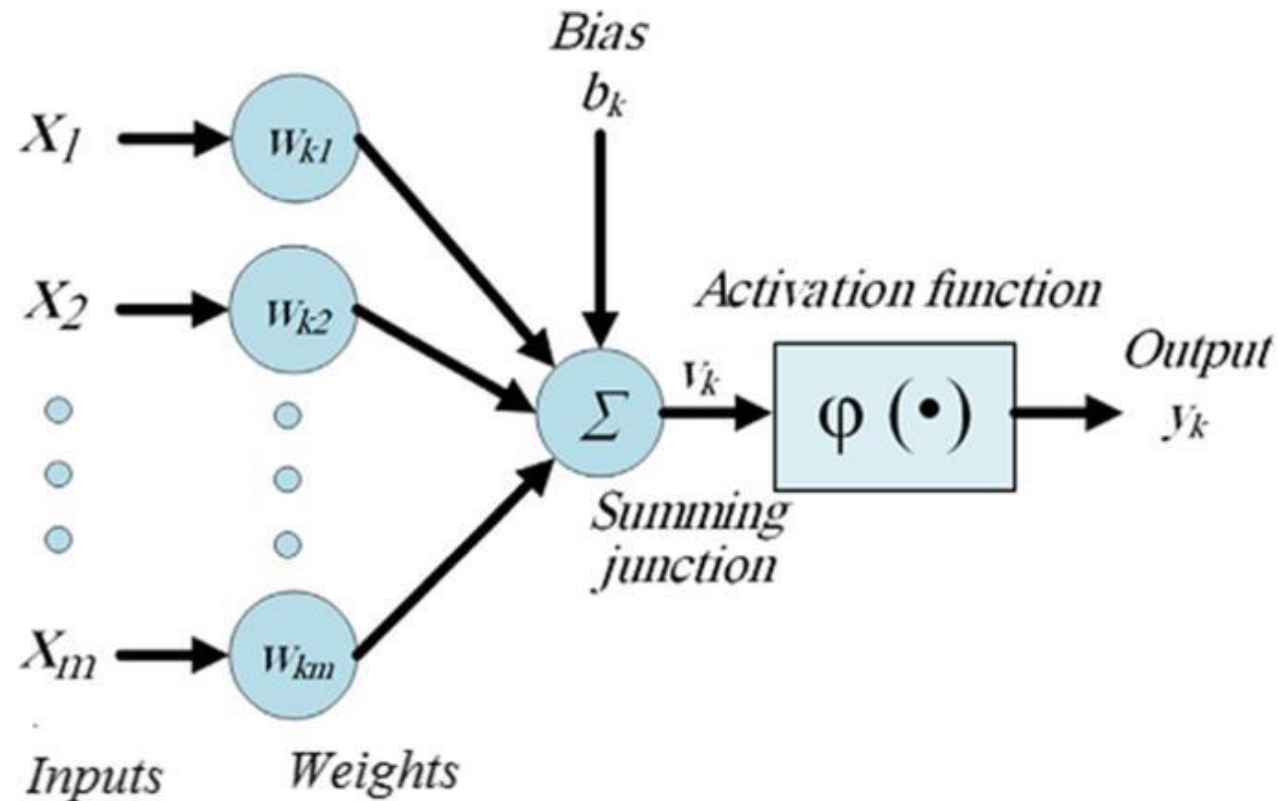
Artificial Neural Network (ANN):

- The term "Artificial neural network" refers to a biologically inspired sub-field of artificial intelligence modeled after the brain.
- An Artificial neural network is usually a computational network based on biological neural networks that construct the structure of the human brain.
- Similar to a human brain has neurons interconnected to each other, artificial neural networks also have neurons that are linked to each other in various layers of the networks. These neurons are known as nodes.

Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

- For example, an acceptable range of output is usually between 0 and 1, or it could be -1 and 1.
- Mathematically, this process is described in the figure,

Basic elements of an ANN



- The output of the neuron, y_k , outcome of some activation function on the value of v_k .
- The values $w_{k1}, w_{k2}, \dots, w_{km}$ are **weights** to determine the strength of **input** $x_1, x_2, \dots, x_m$.

$$v_k = \sum_{j=1}^p w_{kj} x_j$$

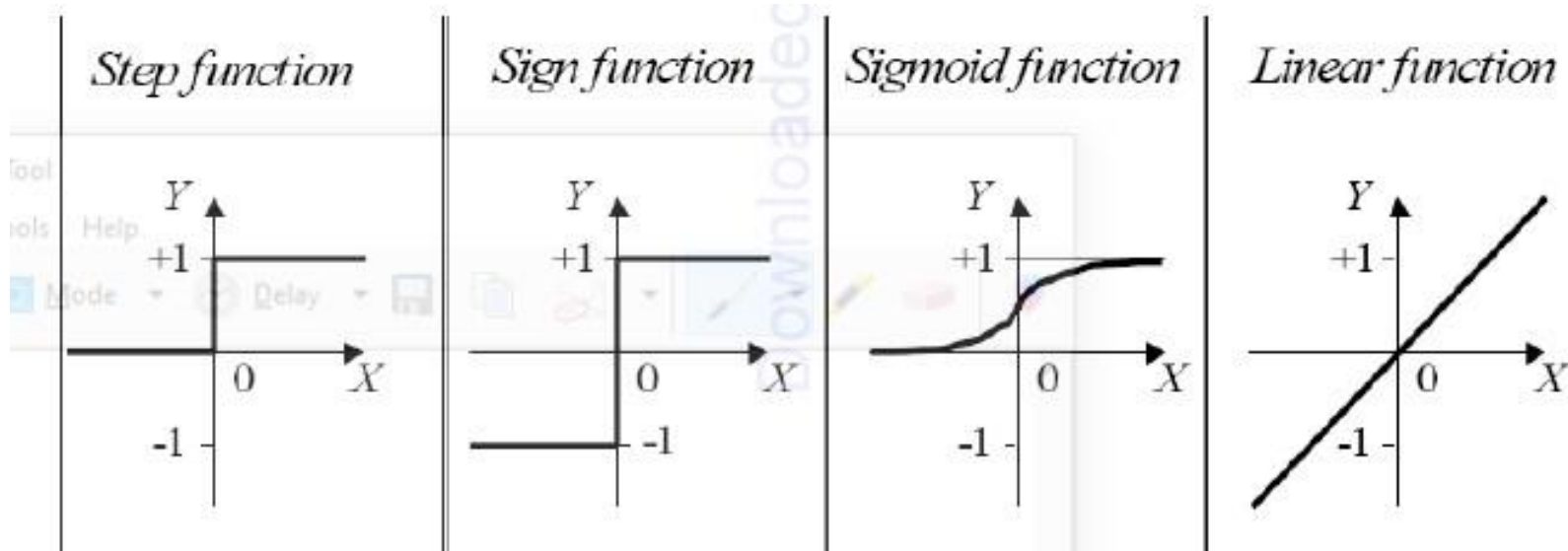
- **Bias** is a trainable parameter that helps the network learn better by shifting the activation function. It enables the model to **capture complex patterns** and improve learning accuracy.
- The neuron computes the weighted sum of the input signals and compares the result with a **threshold value**, ϕ .
- The output can be represented as “1” or “0.” It can also be represented as “1” or “-1” depending on which activation function is used.

Activation function

An activation function f performs a mathematical operation on the signal output.

Types of Activation Function

(where v_k is X and y_k is Y is)



$$Y = \begin{cases} 1, & \text{if } X \geq 0 \\ 0, & \text{if } X < 0 \end{cases}$$

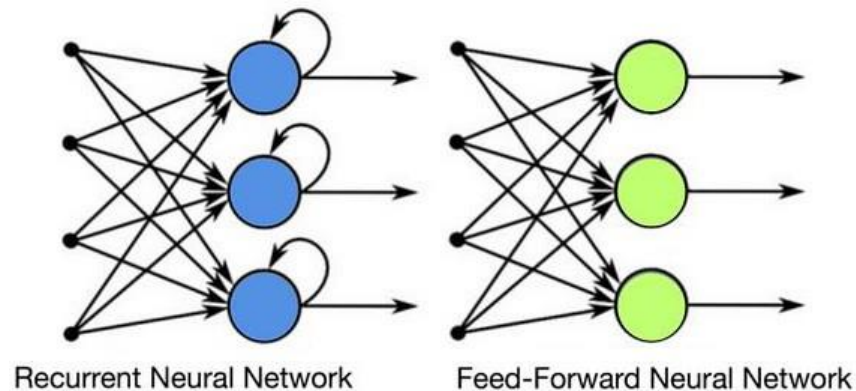
$$Y = \begin{cases} +1, & \text{if } X \geq 0 \\ -1, & \text{if } X < 0 \end{cases}$$

$$Y = 1/(1 + e^{-X})$$

$$Y = X$$

Types of ANN

1. Feed-forward neural network
2. Recurrent neural network(feedback)
 - A feed-forward neural network processes information in a linear manner, moving from the input layer through hidden layers to the output layer without any loops or feedback.
 - Feed-forward networks lack memory and struggle with predicting future events. They only focus on the current input and don't retain any information about the past, except what they learned during training.



- In a Recurrent Neural Network (RNN), information undergoes a cyclical process within a loop.
- When the network makes a decision, it takes into account not only the current input but also integrates knowledge acquired from past inputs.
- This ability to consider and incorporate information from previous time steps allows RNNs to exhibit a form of memory, enabling them to better understand and analyze sequences of data.

a. Single layer feed forward network

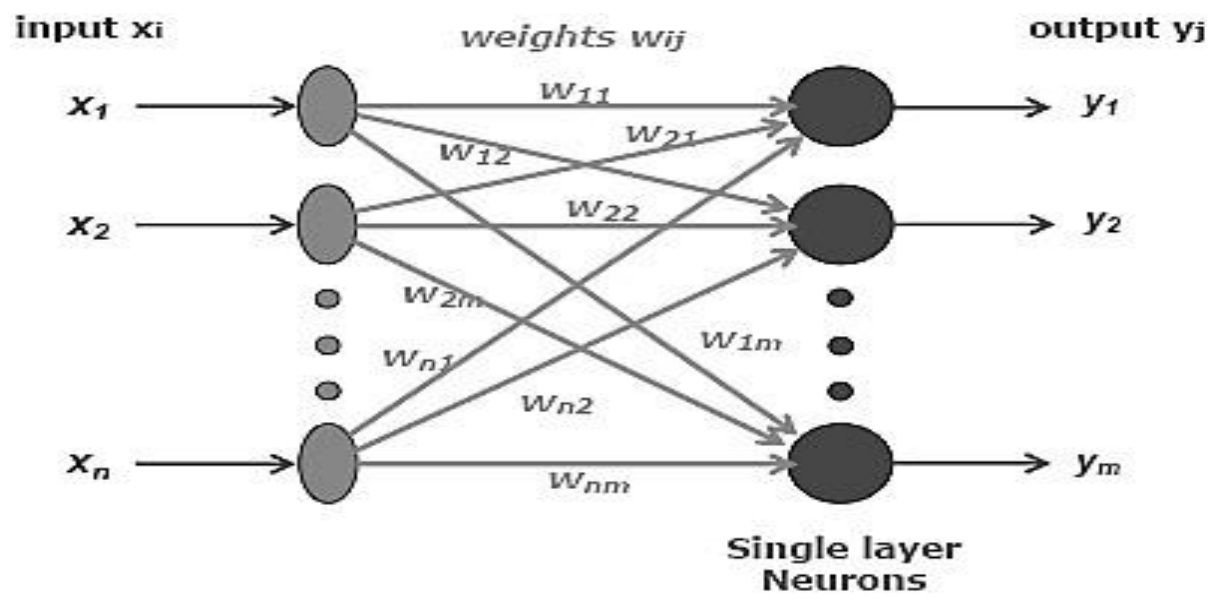


Fig. Single Layer Feed-forward Network

b. Multi layer feed forward network

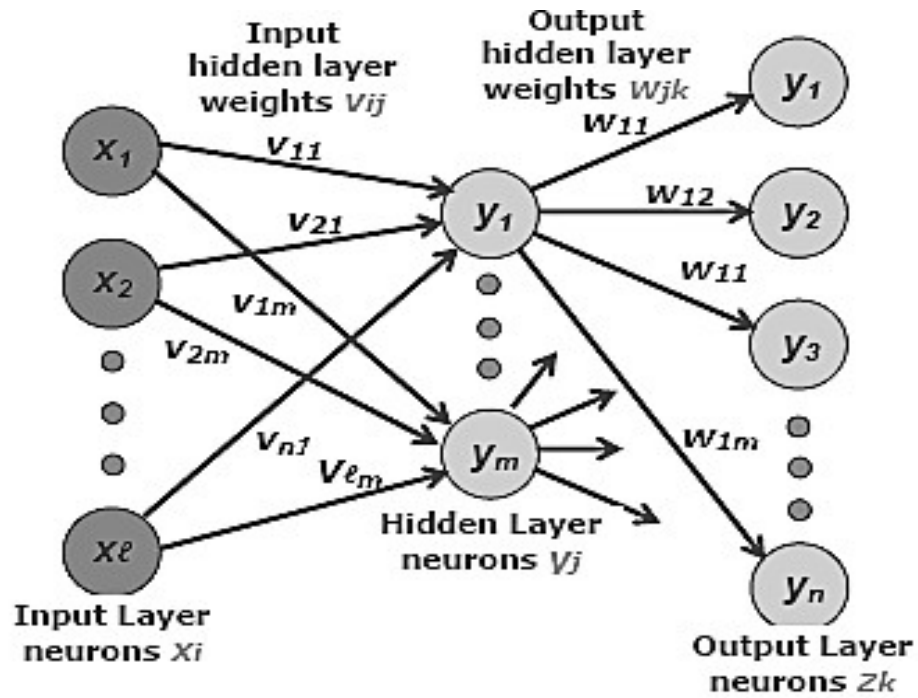


Fig. Multilayer feed-forward network in $(\ell - m - n)$ configuration.

Neural Network Applications

1. Financial Modeling – Predicting the Stock Market

- **Stock Price Prediction:** Neural networks analyze historical stock prices, market indicators, and other financial data to predict future stock prices. Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are particularly useful for this purpose.
- **Algorithmic Trading:** Neural networks make trading decisions based on predicted price movements and market conditions, executing trades automatically to maximize profits.

2. Time Series Prediction – Climate, Weather, Seizures

- **Climate and Weather Forecasting:** Neural networks predict future climate patterns and weather conditions by analyzing historical weather data, satellite images, and other meteorological information.
- **Seizure Prediction:** In healthcare, neural networks analyze EEG signals to predict epileptic seizures, allowing for timely interventions and better management of epilepsy.

3. Computer Games – Intelligent Agents, Chess, Backgammon

- **Intelligent Agents:** Neural networks power intelligent agents in video games, enabling them to learn and adapt to players' actions, creating more challenging and engaging gameplay.
- **Game Playing:** Neural networks, combined with reinforcement learning, have achieved superhuman performance in games like chess, backgammon, and Go by learning optimal strategies through self-play and historical game data.

4. Robotics – Autonomous Adaptable Robots

- **Autonomous Navigation:** Neural networks enable robots to navigate autonomously in complex environments by processing sensor data and making real-time decisions.
- **Adaptability:** Robots equipped with neural networks can adapt to changes in their environment and perform tasks such as object recognition, manipulation, and obstacle avoidance.

5. Pattern Recognition – Speech Recognition, Seismic Activity, Sonar Signals

- **Speech Recognition:** Neural networks convert spoken language into text by recognizing patterns in audio signals. Applications include virtual assistants, transcription services, and language translation.
- **Seismic Activity:** Neural networks analyze seismic data to detect and predict earthquakes, providing early warnings and helping in disaster preparedness.
- **Sonar Signal Processing:** In underwater navigation and exploration, neural networks process sonar signals to detect objects and map the underwater environment.

6. Data Analysis – Data Compression, Data Mining

- **Data Compression:** Neural networks, such as autoencoders, are used for data compression by learning efficient representations of the input data, reducing storage and transmission requirements.
- **Data Mining:** Neural networks identify patterns and extract valuable insights from large datasets in various domains, including marketing, healthcare, and finance.

7. Bioinformatics – DNA Sequencing, Alignment

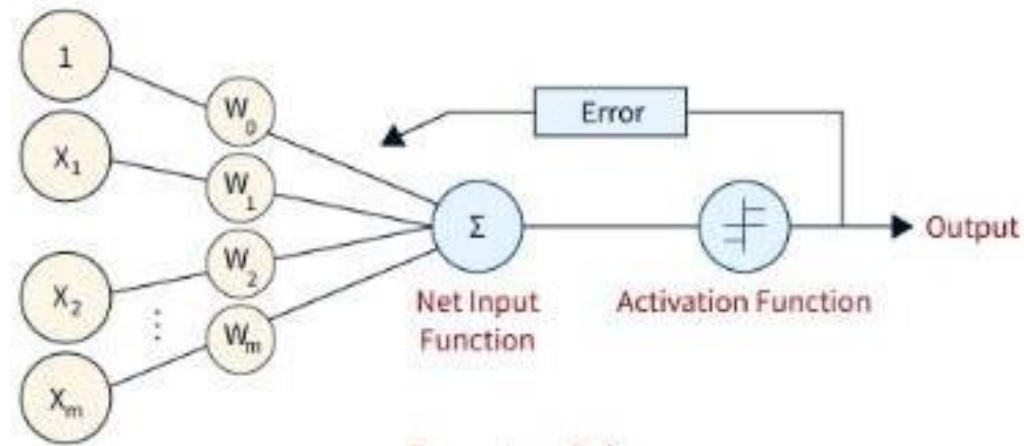
- **DNA Sequencing:** Neural networks analyze genetic data to identify sequences, detect mutations, and understand genetic variations. This aids in research and personalized medicine.
- **Sequence Alignment:** Neural networks align DNA sequences to identify similarities and differences, which is crucial for studying evolutionary relationships, gene function, and genetic disorders.

<u>Aspect</u>	<u>Biological Neural Networks</u>	<u>Artificial Neural Networks</u>
Neurons	Biological cells	Mathematical functions
Learning	Neuroplasticity, Hebbian learning	Supervised learning, backpropagation
Adaptability	Highly adaptable	Adaptable through training on data
Processing	Parallel processing	Both parallel and sequential processing
Complexity	Extremely complex, with billions of neurons and trillions of connections	Varies, but typically less complex than biological networks
Activation	Chemical and electrical signals	Mathematical functions like ReLU, Sigmoid
Structure	Dynamic, with changing synaptic strengths	Static architecture during training, but weights are updated
Input and Output	Sensory inputs and motor outputs	Numerical inputs and outputs, task-dependent

Learning by Neural Network

- **Perceptron Learning:**

- Perceptron is a building block of an Artificial Neural Network.
- Mr. Frank Rosenblatt invented the Perceptron for performing certain calculations to detect input data capabilities or business intelligence.



Perceptron Learning Algorithm:

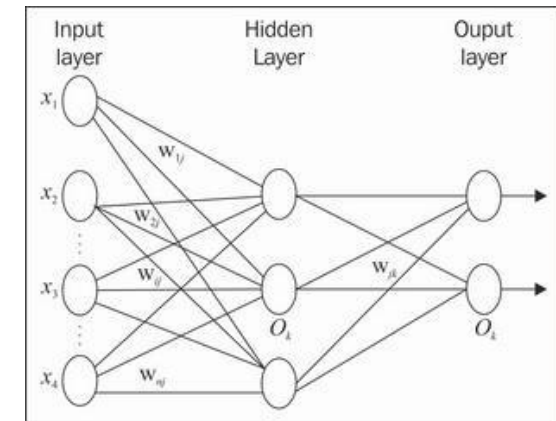
1. Multiply all input values with corresponding weight values and then add them to determine the weighted sum. $\sum w_i * x_i = x_1 * w_1 + x_2 * w_2 + \dots + w_n * x_n$. Add another essential term called bias 'b' to the weighted sum to improve the model performance. $\sum w_i * x_i + b$
 2. An activation function is applied to this weighed sum, producing a binary or a continuous-valued output. $Y = f(\sum w_i * x_i + b)$
 3. The difference between this output and the actual target value is computed to get the error term, E, generally in terms of mean squared error. The steps up to this form the forward propagation part of the algorithm. $E = (Y - Y_{\text{actual}})^2$
- We optimize this error (loss function) using an optimization algorithm. Generally, some form of gradient descent algorithm is used to find the optimal values of the hyper parameters like learning rate, weight, Bias, etc. This step forms the backward propagation part of the algorithm.

Types of Perceptron models

- **Single Layer Perceptron model:** it consists of a feed-forward network and includes a threshold transfer inside the model.
- The main objective of the single-layer perceptron model is to analyze the linearly separable objects with binary outcomes.
- **Multi-Layered Perceptron model:** It is mainly similar to a single-layer perceptron model but has more hidden layers.

- **Backpropagation (BP) Learning :**

- The **backpropagation** algorithm learns the classification model by training a multilayer feed-forward neural network.
- The generic architecture of the neural network for BP is shown in the following diagrams, with one input layer, some hidden layers, and one output layer.
- Each layer contains some units or perceptron. Each unit might be linked to others by weighted connections. The values of the weights are initialized before the training. The number of units in each layer, number of hidden layers, and the connections will be empirically defined at the very start.
- Backpropagation is a supervised learning algorithm. It is used in training artificial neural networks by adjusting the weights based on the error of the output compared to the expected output.



Training Algorithm :

Input:

- D , a data set consisting of the training tuples and their associated target values;
- l , the learning rate;
- $network$, a multilayer feed-forward network.

Output: A trained neural network.

Method:

- (1) Initialize all weights and biases in $network$;
- (2) **while** terminating condition is not satisfied {
- (3) **for** each training tuple X in D {
- (4) // Propagate the inputs forward:
- (5) **for** each input layer unit j {
- (6) $O_j = I_j$; // output of an input unit is its actual input value
- (7) **for** each hidden or output layer unit j {
- (8) $I_j = \sum_i w_{ij} O_i + \theta_j$; // compute the net input of unit j with respect to the previous layer, i
- (9) $O_j = \frac{1}{1+e^{-I_j}}$; } // compute the output of each unit j

```

(10) // Backpropagate the errors:
(11) for each unit  $j$  in the output layer
(12)      $Err_j = O_j(1 - O_j)(T_j - O_j)$ ; // compute the error
(13) for each unit  $j$  in the hidden layers, from the last to the first hidden layer
(14)      $Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$ ; // compute the error with respect to
        the next higher layer,  $k$ 
(15) for each weight  $w_{ij}$  in network {
(16)      $\Delta w_{ij} = (l)Err_j O_i$ ; // weight increment
(17)      $w_{ij} = w_{ij} + \Delta w_{ij}$ ; } // weight update
(18) for each bias  $\theta_j$  in network {
(19)      $\Delta \theta_j = (l)Err_j$ ; // bias increment
(20)      $\theta_j = \theta_j + \Delta \theta_j$ ; } // bias update
(21) } }

```

Example: Consider a multilayer feed-forward neural network given below . Let the learning rate be 0.9. Assume initial weight and bias values of the network are given in the table below, along with the first training tuple, $X = (1, 0, 1)$, whose class label is 1. Show weight and bias updates by using back-propagation algorithm.

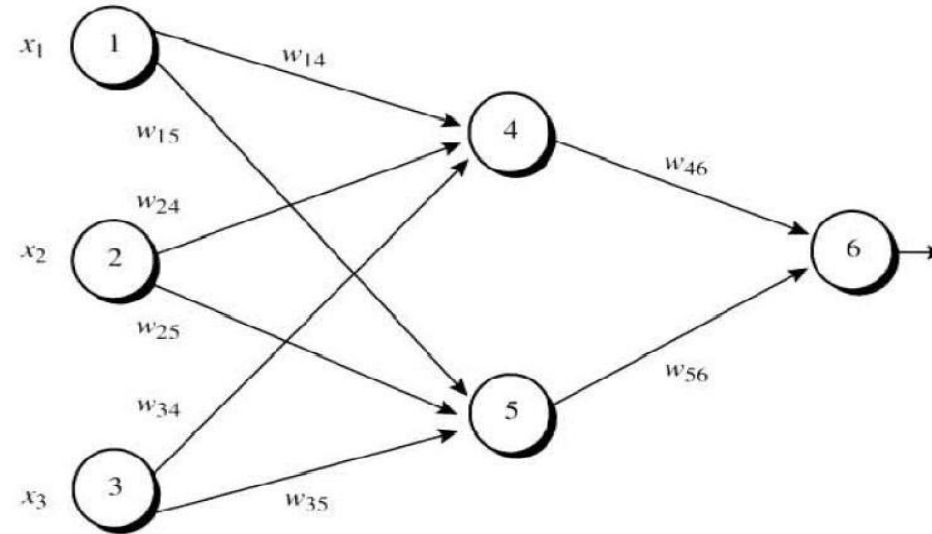


Figure: An example of a multilayer feed-forward neural network.

x_1	x_2	x_3	w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	θ_4	θ_5	θ_6
1	0	1	0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

Table : Initial input, weight, and bias values.

Now calculate the output for each nodes as follows:

For input layer:

$$O_1 = x_1 = 1, O_2 = x_2 = 0, O_3 = x_3 = 1$$

Table: The net input and output calculations

Unit j	Net input, I_j	Output, O_j
4	$0.2 + 0 - 0.5 - 0.4 = -0.7$	$1/(1 + e^{0.7}) = 0.332$
5	$-0.3 + 0 + 0.2 + 0.2 = 0.1$	$1/(1 + e^{-0.1}) = 0.525$
6	$(-0.3)(0.332) - (0.2)(0.525) + 0.1 = -0.105$	$1/(1 + e^{0.105}) = 0.474$

Then calculate the error at each node:

Table : Calculation of the error at each node.

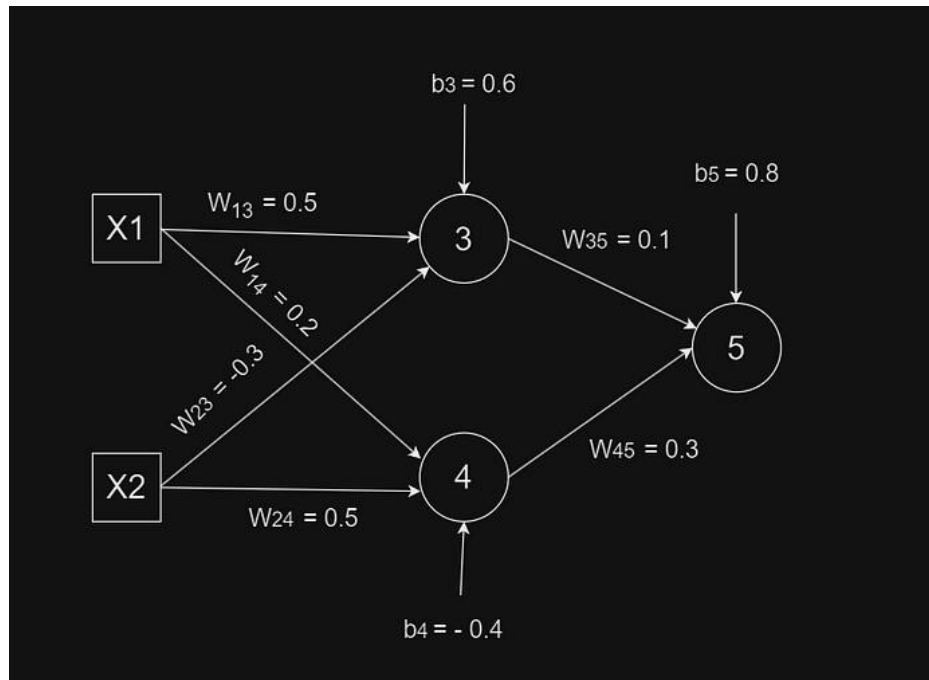
Unit j	Err j
6	$(0.474)(1 - 0.474)(1 - 0.474) = 0.1311$
5	$(0.525)(1 - 0.525)(0.1311)(-0.2) = -0.0065$
4	$(0.332)(1 - 0.332)(0.1311)(-0.3) = -0.0087$

Now update the weight and bias as follows:

Table : Calculation for weight and bias updating.

Weight or bias	New value
w_{46}	$-0.3 + (0.9)(0.1311)(0.332) = -0.261$
w_{56}	$-0.2 + (0.9)(0.1311)(0.525) = -0.138$
w_{14}	$0.2 + (0.9)(-0.0087)(1) = 0.192$
w_{15}	$-0.3 + (0.9)(-0.0065)(1) = -0.306$
w_{24}	$0.4 + (0.9)(-0.0087)(0) = 0.4$
w_{25}	$0.1 + (0.9)(-0.0065)(0) = 0.1$
w_{34}	$-0.5 + (0.9)(-0.0087)(1) = -0.508$
w_{35}	$0.2 + (0.9)(-0.0065)(1) = 0.194$
θ_6	$0.1 + (0.9)(0.1311) = 0.218$
θ_5	$0.2 + (0.9)(-0.0065) = 0.194$
θ_4	$-0.4 + (0.9)(-0.0087) = -0.408$

Q. Consider a multilayer feed-forward neural network given below. Let the learning rate be 0.5. Assume initial values of weights and biases as given in the table below. Train the network for the training tuples (1, 1, 0) and (0, 1, 1), where last number is target output. Show weight and bias updates by using back-propagation algorithm. Assume that sigmoid activation function is used in the network.



Solution:

Step 1: Initialization

a. Inputs and Outputs for 2 iterations:

x1	x2	Output(τ)
1	1	0
0	1	1

b. Initial weight and bias values:

W13	W14	W23	W24	W35	W45	b3	b4	b5
0.5	0.2	-0.3	0.5	0.1	0.3	0.6	-0.4	0.8

c. Learning rate (η) = 0.5

d. Activation function = Sigmoid function

$$\text{(i.e. } \sigma = \frac{1}{1 + e^{-x}} \text{)}$$

Step 2: The termination condition for our algorithm, according to the question, is when we complete 2 iterations.

Iteration 1

Input and Output values:

X1	X2	Output(τ)
1	1	0

Weight (w) and bias(b) values:

W13	W14	W23	W24	W35	W45	b3	b4	b5
0.5	0.2	-0.3	0.5	0.1	0.3	0.6	-0.4	0.8

a. Calculate the input(I) and output(O) for each layer

-For input layer

$$O_1 = I_1 = 1$$

$$O_2 = I_2 = 1$$

-For hidden layer

$$\begin{aligned} I_3 &= O_1 \times W_{13} + O_2 \times W_{23} + b_3 \\ &= 1 \times 0.5 + 1 \times (-0.3) + 0.6 \\ &= 0.5 - 0.3 + 0.6 \\ &= 0.8 \end{aligned}$$

$$O_3 = \frac{1}{1 + e^{-I_3}} = \frac{1}{1 + e^{-0.8}} = 0.69$$

$$\begin{aligned} I_4 &= O_1 \times W_{14} + O_2 \times W_{24} + b_4 \\ &= 1 \times 0.2 + 1 \times 0.5 + (-0.4) \\ &= 0.2 + 0.5 - 0.4 \\ &= 0.3 \end{aligned}$$

$$O_4 = \frac{1}{1 + e^{-I_4}} = \frac{1}{1 + e^{-0.3}} = 0.5744$$

-For output layer

$$\begin{aligned} I_5 &= O_3 \times W_{35} + O_4 \times W_{45} + b_5 \\ &= 0.69 \times 0.1 + 0.5744 \times 0.3 + 0.8 \\ &= 0.0690 + 0.1723 + 0.8 \\ &= 1.0413 \end{aligned}$$

$$O_5 = \frac{1}{1 + e^{-I_5}} = \frac{1}{1 + e^{-1.0413}} = 0.7391$$

b. Calculation of error at each node:

Error calculation formula for output layer: $Err_j = O_j(1 - O_j)(T_j - O_j)$

- **For output layer:**

$$\begin{aligned}
 Err_5 &= O_5 (1 - O_5) (\tau - O_5) \\
 &= 0.7391 (1 - 0.7391) (0 - 0.7391) \\
 &= 0.7391 (0.2609) (- 0.7391) \\
 &= - 0.1425
 \end{aligned}$$

Calculation formula for hidden layer:

$$Err_j = O_j(1 - O_j) \sum_k Err_k w_{jk}$$

- **For hidden layer:**

$$\begin{aligned}
 Err_4 &= O_4 (1 - O_4) (Err_5 \times W_{45}) \\
 &= 0.5744 (1 - 0.5744) (- 0.1425 \times 0.3) \\
 &= 0.5744 (0.4256) (- 0.04275) \\
 &= - 0.010451
 \end{aligned}$$

$$\begin{aligned}
 Err_3 &= O_3 (1 - O_3) (Err_5 \times W_{35}) \\
 &= 0.69 (1 - 0.69) (- 0.1425 \times 0.1) \\
 &= 0.69 (0.31) (- 0.1425) \\
 &= - 0.030481
 \end{aligned}$$

c. Update the weight

$$W_{ij}(\text{new}) = W_{ij}(\text{old}) + I \times O_i \times \text{Err}_j$$

$$W_{45}(\text{new}) = W_{45}(\text{old}) + I \times O_4 \times \text{Err}_5 = 0.3 + 0.5 \times 0.5744 \times (-0.1425) = 0.2591$$

$$W_{35}(\text{new}) = W_{35}(\text{old}) + I \times O_3 \times \text{Err}_5 = 0.1 + 0.5 \times 0.69 \times (-0.1425) = 0.0508$$

$$W_{24}(\text{new}) = W_{24}(\text{old}) + I \times O_2 \times \text{Err}_4 = 0.5 + 0.5 \times 1 \times (-0.010451) = 0.494775$$

$$W_{14}(\text{new}) = W_{14}(\text{old}) + I \times O_1 \times \text{Err}_4 = 0.2 + 0.5 \times 1 \times (-0.010451) = 0.194774$$

$$W_{23}(\text{new}) = W_{23}(\text{old}) + I \times O_2 \times \text{Err}_3 = -0.3 + 0.5 \times 1 \times (-0.030481) = -0.315240$$

$$W_{13}(\text{new}) = W_{13}(\text{old}) + I \times O_1 \times \text{Err}_3 = 0.5 + 0.5 \times 1 \times (-0.030481) = 0.484759$$

d. Update the bias value

$$b_j = b_{j(\text{old})} + I \times \text{Err}_j$$

$$b_3 = b_{3(\text{old})} + I \times \text{Err}_3 = 0.6 + 0.5 \times (-0.030481) = 0.584759$$

$$b_4 = b_{4(\text{old})} + I \times \text{Err}_4 = -0.4 + 0.5 \times (-0.010451) = -0.405225$$

$$b_5 = b_{5(\text{old})} + I \times \text{Err}_5 = 0.8 + 0.5 \times (-0.1425) = 0.7288$$

Iteration 2

Input and output values:

x1	x2	Output(τ)
0	1	1

Weight(W) and bias(b) values:

W13	W14	W23	W24	W35	W45	b3	b4	b5
0.4848	0.1948	-0.315	0.4948	0.0508	0.2591	0.5848	-0.405	0.7288

a. Calculate the input(I) and output(O) for each layer

-For input layer

$$O_1 = I_1 = 0$$

$$O_2 = I_2 = 1$$

-For hidden layer:

$$\begin{aligned} I_3 &= O_1 \times W_{13} + O_2 \times W_{23} + b_3 \\ &= 0 \times 0.4848 + 1 \times (-0.315) + 0.5848 \\ &= 0 - 0.315 + 0.5848 \\ &= 0.2698 \end{aligned}$$

$$O_3 = \frac{1}{1 + e^{-I_3}} = \frac{1}{1 + e^{-0.2698}} = 0.567044$$

$$\begin{aligned} I_4 &= O_1 \times W_{14} + O_2 \times W_{24} + b_4 \\ &= 0 \times 0.1948 + 1 \times 0.4948 + (-0.405) \\ &= 0 + 0.4948 - 0.405 \\ &= 0.0898 \end{aligned}$$

$$O_4 = \frac{1}{1 + e^{-I_4}} = \frac{1}{1 + e^{-0.0898}} = 0.522435$$

-For output layer:

$$\begin{aligned} I_5 &= O_3 \times W_{35} + O_4 \times W_{45} + b_5 \\ &= 0.567044 \times 0.0508 + 0.522435 \times 0.2591 + 0.7288 \\ &= 0.892969 \end{aligned}$$

$$O_5 = \frac{1}{1 + e^{-I_5}} = \frac{1}{1 + e^{-0.892969}} = 0.709502$$

b. Calculation of error at each node:

- **For output layer:**

$$\begin{aligned} \text{Err}_5 &= O_5 (1 - O_5) (\tau - O_5) \\ &= 0.709502 (1 - 0.709502) (1 - 0.709502) \\ &= 0.709502 (0.290498) (0.290498) \\ &= 0.059874 \end{aligned}$$

- **For hidden layer:**

$$\begin{aligned} \text{Err}_4 &= O_4 (1 - O_4) (\text{Err}_5 \times W_{45}) \\ &= 0.522435 (1 - 0.522435) (0.059874 \times 0.2591) \\ &= 0.003871 \end{aligned}$$

$$\begin{aligned} \text{Err}_3 &= O_3 (1 - O_3) (\text{Err}_5 \times W_{35}) \\ &= 0.567044 (1 - 0.567044) (0.059874 \times 0.0508) \\ &= 0.567044 (0.432956) (0.003042) \\ &= 0.000747 \end{aligned}$$

c. Update the weight

$$W_{45}(\text{new}) = W_{45}(\text{old}) + I \times O_4 \times \text{Err}_5 = 0.2591 + 0.5 \times 0.522435 \times 0.059874 = 0.274740$$

$$W_{35}(\text{new}) = W_{35}(\text{old}) + I \times O_3 \times \text{Err}_5 = 0.0508 + 0.5 \times 0.567044 \times 0.059874 = 0.067776$$

$$W_{24}(\text{new}) = W_{24}(\text{old}) + I \times O_2 \times \text{Err}_4 = 0.4948 + 0.5 \times 1 \times 0.003871 = 0.496735$$

$$W_{14}(\text{new}) = W_{14}(\text{old}) + I \times O_1 \times \text{Err}_4 = 0.1948 + 0.5 \times 0 \times 0.003871 = 0.1948$$

$$W_{23}(\text{new}) = W_{23}(\text{old}) + I \times O_2 \times \text{Err}_3 = -0.315 + 0.5 \times 1 \times 0.000747 = -0.314626$$

$$W_{13}(\text{new}) = W_{13}(\text{old}) + I \times O_1 \times \text{Err}_3 = 0.4848 + 0.5 \times 0 \times 0.000747 = 0.4848$$

Our calculation stops here.

The updated weight and bias values after second iterations are:

d. Update the bias value

$$b_3 = b_{3(\text{old})} + I \times \text{Err}_3 = 0.5848 + 0.5 \times 0.000747 = 0.585173$$

$$b_4 = b_{4(\text{old})} + I \times \text{Err}_4 = -0.405 + 0.5 \times 0.003871 = -0.403064$$

$$b_5 = b_{5(\text{old})} + I \times \text{Err}_5 = 0.7288 + 0.5 \times 0.059874 = 0.758737$$

W13	W14	W23	W24	W35	W45	b3	b4	b5
0.4848	0.1948	-0.315	0.4967	0.0678	0.2747	0.5852	-0.403	0.7587

▪ **Hebbian Learning:**

- Donald Hebb developed it in 1949 as an unsupervised learning algorithm in the neural network.
- Hebbian rule works by updating the weights between neurons in the neural network for each training sample.

Hebbian Learning Algorithm:

Step 1: Set all weights to zero, $w_i = 0$ for $i=1$ to n , and bias to zero.

Step 2 : For each input training input , S (input vector), with target output , t (target output pair), set the activations of input units: $X_i = S_i$ for $i = 1$ to n .

Step 3 : Set the corresponding output value to the output neuron, i.e. $y = t$.

Step 4 : Update weight and bias by applying Hebb rule for all $i = 1$ to n :

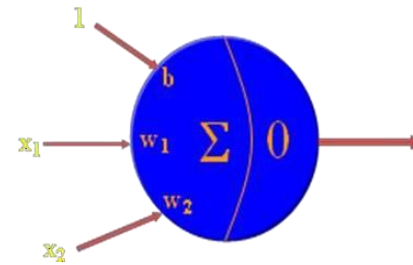
$$w_i (\text{new}) = w_i (\text{old}) + x_i y$$

$$b (\text{new}) = b (\text{old}) + y$$

Example: Construct a Hebb Net which performs like an AND function, that is, only when both features are “active” will the data be in the target class.

Training set (with the bias input always at 1):

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1



Solution:

Step 1 : Set weight and bias to zero, $w_1=w_2=b=0$.

Step 2 : Set input vector $X_i = S_i$ for $i = 1$ to 4.

$$X_1 = [1 \ 1 \ 1]$$

$$X_2 = [1 \ -1 \ 1]$$

$$X_3 = [-1 \ 1 \ 1]$$

$$X_4 = [-1 \ -1 \ 1]$$

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1

Step 3 : Output value is set to $y = t$.

Step 4 : Modifying weights using Hebbian Rule : $W_i \text{ (new)} = w_i \text{ (old)} + x_i y$

First input $(x_1 \ x_2 \ b) = [1 \ 1 \ 1]$ and target of 1:

$$W_i \text{ (new)} = w_i \text{ (old)} + x_i y;$$

$$W_1 \text{ (new)} = w_1 \text{ (old)} + x_1 y = 0 + 1 = 1$$

$$W_2 \text{ (new)} = w_2 \text{ (old)} + x_2 y = 0 + 1 = 1$$

$$b \text{ (new)} = b \text{ (old)} + y = 0 + 1 = 1$$

second input, $(x_1 \ x_2 \ b)=[1 \ -1 \ 1]$ and target of -1:

$$W_i(\text{new}) = w_i(\text{old}) + x_i y ;$$

$$W_1(\text{new}) = w_1(\text{old}) + x_1 y = 1 + 1(-1) = 0$$

$$W_2(\text{new}) = w_2(\text{old}) + x_2 y = 1 + (-1)(-1) = 2$$

$$b(\text{new}) = b(\text{old}) + y = 1 + (-1) = 0$$

Third input , $(x_1 \ x_2 \ b)=[-1 \ 1 \ 1]$ and target of -1

$$W_i(\text{new}) = w_i(\text{old}) + x_i y ;$$

$$W_1(\text{new}) = w_1(\text{old}) + x_1 y = 0 + (-1)(-1) = 1$$

$$W_2(\text{new}) = w_2(\text{old}) + x_2 y = 2 + 1(-1) = 1$$

$$b(\text{new}) = b(\text{old}) + y = 1 + (-1) = 1$$

Fourth input , $(x_1 \ x_2 \ b)=[-1 \ -1 \ 1]$ and target of -1

$$W_i(\text{new}) = w_i(\text{old}) + x_i y ;$$

$$W_1(\text{new}) = w_1(\text{old}) + x_1 y = 1 + (-1)(-1) = 2$$

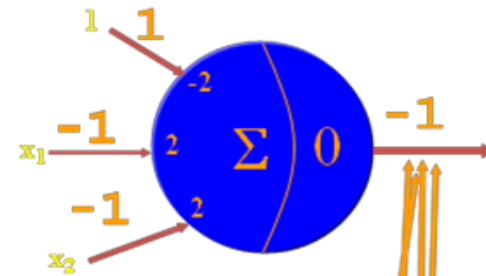
$$W_2(\text{new}) = w_2(\text{old}) + x_2 y = 1 + (-1)(-1) = 2$$

$$b(\text{new}) = b(\text{old}) + y = -1 + (-1) = -2$$

Final Neuron:

- **This neuron works:**

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	1
-1	-1	1	-1



$$\begin{aligned} 1*2 + 1*2 + 1*(-2) &= 2 > 0 \\ (-1)*2 + 1*2 + 1*(-2) &= -2 < 0 \\ 1*2 + (-1)*2 + 1*(-2) &= -2 < 0 \\ (-1)*2 + (-1)*2 + 1*(-2) &= -6 < 0 \end{aligned}$$

Overview of Deep Learning (DL) Architectures

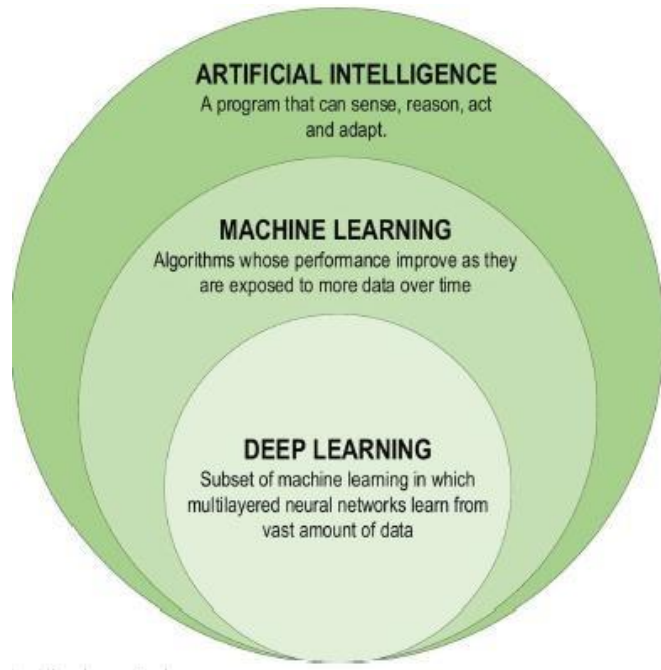


Figure: Deep learning family

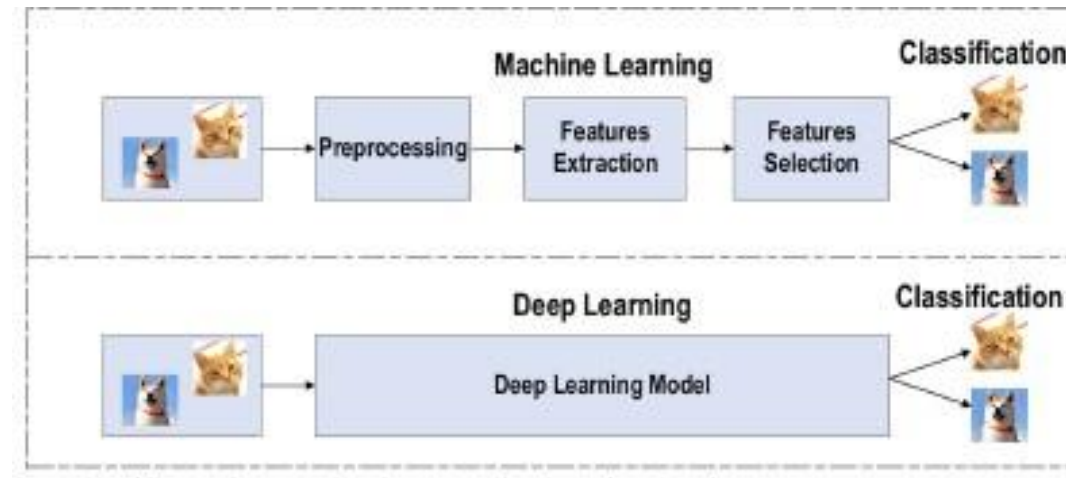


Figure: The difference between deep learning and traditional machine learning

When to apply deep learning

Machine intelligence is useful in many situations which is equal or better than human experts in some cases, meaning that DL can be a solution to the following problems:

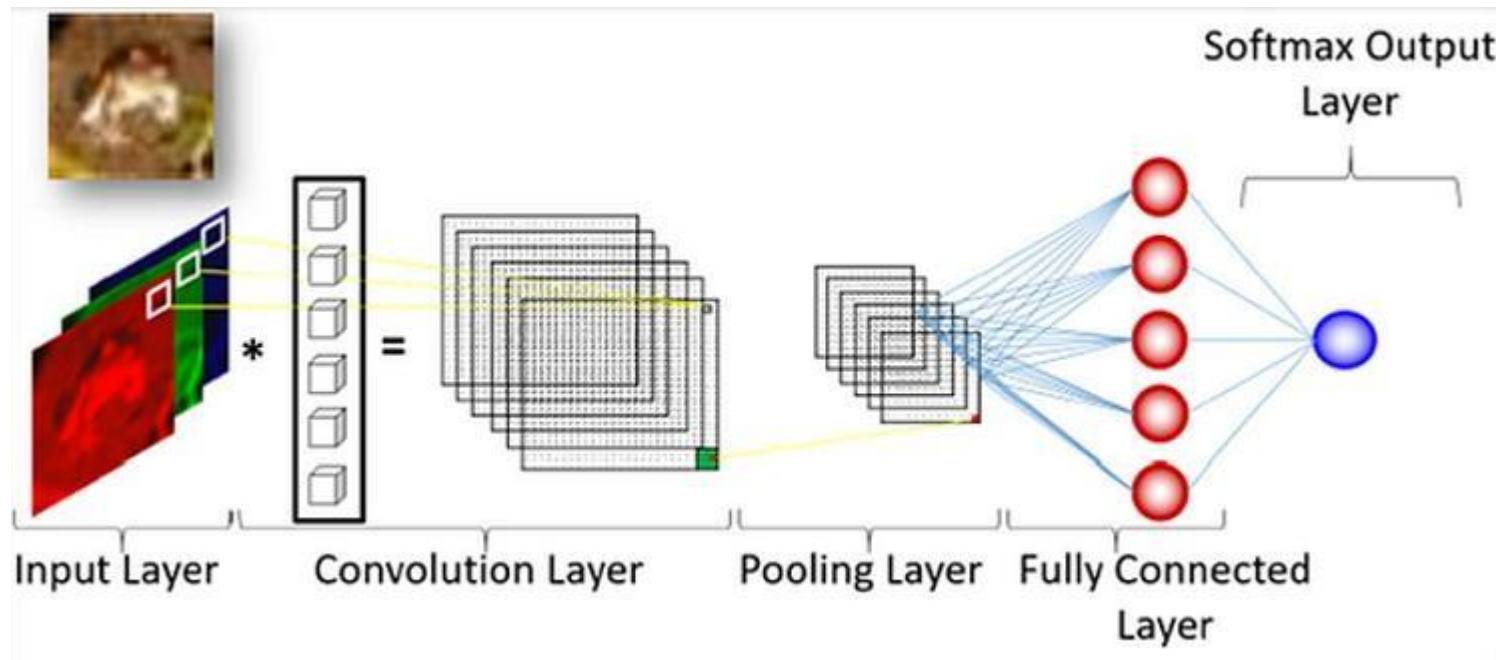
- Cases where human experts are not available.
- Cases where humans are unable to explain decisions made using their expertise (language understanding, medical decisions, and speech recognition).
- Cases where the problem solution updates over time (price prediction, stock preference, weather prediction, and tracking).
- Cases where solutions require adaptation based on specific cases (personalization, biometrics).
- Cases where size of the problem is extremely large and exceeds our inadequate reasoning abilities (sentiment analysis, matching ads to Facebook, calculation webpage ranks).

Why deep learning?

- 1. Universal Learning Approach:** Because DL has the ability to perform in approximately all application domains, it is sometimes referred to as universal learning.
- 2. Robustness:** In general, precisely designed features are not required in DL techniques. Instead, the optimized features are learned in an automated fashion related to the task under consideration. Thus, robustness to the usual changes of the input data is attained.
- 3. Generalization:** Different data types or different applications can use the same DL technique, an approach frequently referred to as transfer learning (TL) which explained in the latter section. Furthermore, it is a useful approach in problems where data is insufficient.
- 4. Scalability:** DL is highly scalable.

Convolutional Neural Networks:

- Convolutional Neural Networks (CNN) are artificial neural networks designed to process and analyze data with grid-like topological structures, such as images and videos.
- CNN as a multi-layered filter that processes images to extract meaningful features and make inference predictions.



CNN layers can be divided into following main parts:

- **Input Layer:** The input layer receives the raw image data. An image is typically represented as a 3D matrix of pixel values (width x height x channels).
- **Convolutional Layers:** These layers apply convolutional operations to the input. A convolution operation involves sliding a filter (kernel) over the input image and computing the dot product between the filter and the local region of the image it covers. This operation extracts features such as edges, textures, and patterns.
- **Pooling Layers:** Pooling (e.g., max pooling or average pooling) reduces the spatial dimensions (width and height) of the feature maps, retaining the most important information and reducing computational complexity.
- **Fully Connected Layers:** After several convolutional and pooling layers, the feature maps are flattened into a 1D vector and passed through fully connected (dense) layers. These layers perform the final classification based on the features extracted by the convolutional layers.

- **Output Layer:** The output layer typically uses a soft max activation function for classification tasks, providing the probability distribution over the possible classes.
- In summary, CNN is a neural network designed to process structured data like images. Its working principle involves applying a series of filters or kernel functions to the image to gradually extract more complex features. Then, through pooling layers, the spatial dimensions are reduced to prevent overfitting. Finally, the output goes through fully connected layers for the final prediction.

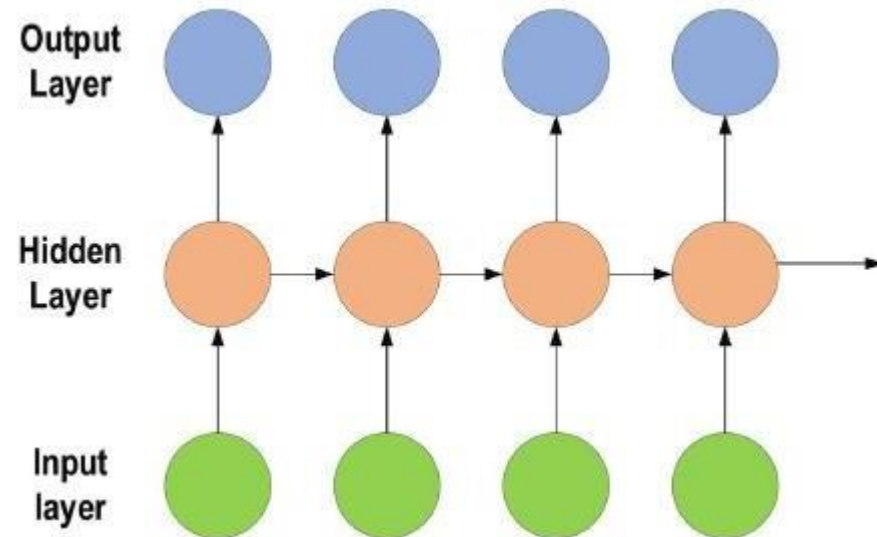
Benefits of employing CNNs

The benefits of using CNNs over other traditional neural networks in the computer vision environment are listed as follows:

- The main reason to consider CNN is the weight sharing feature, which reduces the number of trainable network parameters and in turn helps the network to enhance generalization and to avoid overfitting.
- Concurrently learning the feature extraction layers and the classification layer causes the model output to be both highly organized and highly reliant on the extracted features.
- Large-scale network implementation is much easier with CNN than with other neural networks.

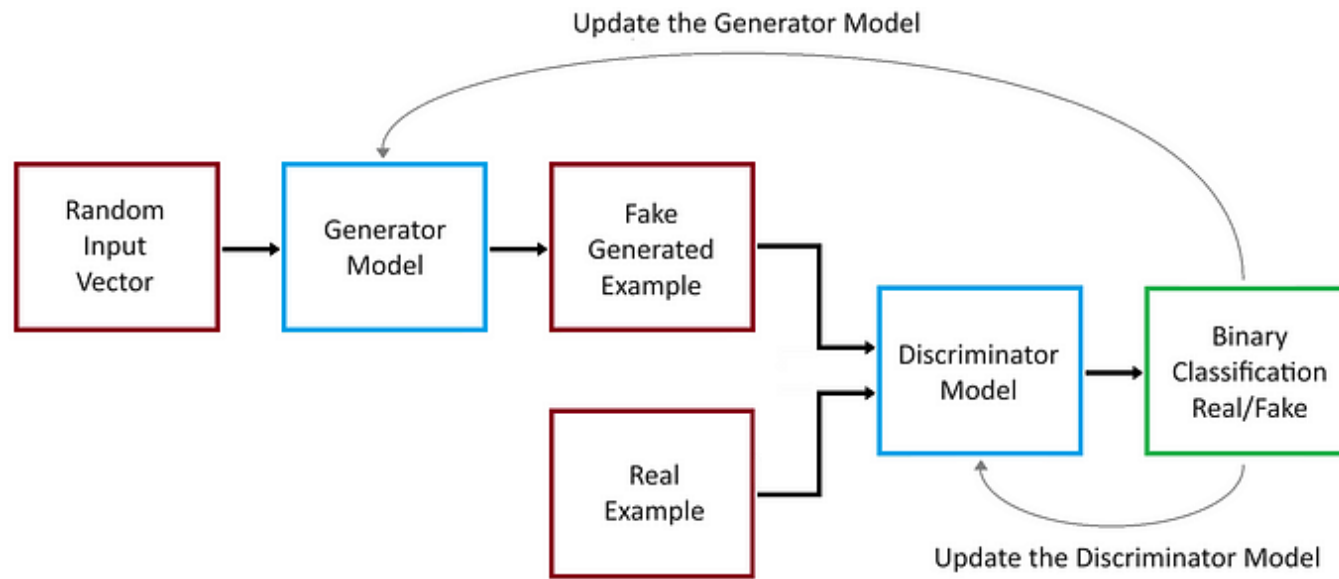
Recurrent Neural Networks (RNNs):

- Recurrent Neural Networks (RNN) are artificial neural networks designed to process sequential data such as time series, speech, and natural language.
- RNN as a conveyor belt that processes information one element at a time, “remembering” the information from the previous element to make predictions for the next element.
- Thus, it is possible to consider the RNN as a unit of short-term memory, where x represents the input layer, y is the output layer, and s represents the state (hidden) layer.



- An RNN can be divided into three main parts: the input layer, the recurrent layer, and the output layer.
- **Input Layer:** The input layer receives the input information at each time step, such as a word in a sequence.
- **Recurrent Layer:** The recurrent layer processes the information from the input layer, utilizing the recurrent connection to “remember” the information from the previous time step. The recurrent layer consists of a set of neurons, each neuron having a recurrent connection to itself and a connection to the current input at the time step.
- **Output Layer:** The output layer generates predictions based on the information processed by the recurrent layer. In the case of generating the next word in a sequence, the output layer predicts the word most likely to appear after the previous word in the sequence.

Generative Adversarial Network (GAN):



- GANs are a **class of neural networks** used for **unsupervised learning**.
- They are capable of generating **realistic images, videos, and voice outputs**.
- A GAN consists of two networks trained together using **adversarial training**:
 1. **Generator**: Takes random noise and generates synthetic data resembling real data.
 2. **Discriminator**: Distinguishes between real and generated data, outputting a probability score.
- The **Generator improves over time** by learning to create more realistic data that can **fool the Discriminator**.

